

Closing the Gap on Knapsack

Fine-Grained Lower Bounds, Near-Quadratic Algorithms, and Learning-Augmented
Scheduling

Week 8 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Why look beyond the syllabus?

- This week you built an $O(nW)$ DP for 0/1 Knapsack and proved it is only **pseudo-polynomial**: polynomial in the *numeric value* W , but exponential in the *number of bits* needed to write W down.
- Two questions a curious student should ask next – both are recently (2017–2024) resolved or actively developing research threads:
 - ① Is $O(nW)$ actually close to the **best possible** in terms of the *numeric* capacity, or could a smarter algorithm beat it substantially?
 - ② This week's DP is **offline**: it assumes every item/job is known in advance. What happens when jobs arrive one at a time, with the future unknown?

How this supplement is different from a survey

Every claim below is sourced to a specific, verifiable paper (venue and year given). Where a bound looks suspiciously precise, that precision is the actual point – these are hard-won constants from real proofs, not rounding for slides.

Recap: the bound you proved this week

This week's lecture notes

0/1 Knapsack's DP runs in $O(nW)$ time and space – polynomial in the *value* W , but W itself takes only $O(\log W)$ bits to write down, so the algorithm is exponential in input *size*. This is **pseudo-polynomial**, and Knapsack is NP-hard, so no genuinely polynomial (in bit-length) algorithm is believed to exist.

- That settles the question of whether Knapsack is polynomial in the input *size*. It says nothing about whether $O(nW)$ is the best possible algorithm that is polynomial in the *numeric value* W – a narrower, sharper question that fine-grained complexity theory answers precisely.

The conditional lower bound

Write $w_{\max}$ for the largest item weight (so $W \leq n w_{\max}$; the DP's real cost is naturally stated in terms of n and $w_{\max}$).

Theorem (Cygan, Mucha, Węgrzycki & Włodarczyk, 2017; independently Künnemann, Paturi & Schneider, 2017)

Under the $(\min, +)$ -**convolution hypothesis**, 0/1 Knapsack has *no* $O((n + w_{\max})^{2-\delta})$ -time algorithm for any $\delta > 0$.

- $(\min, +)$ -convolution: given arrays A, B of length N , compute $C[k] = \min_i (A[i] + B[k - i])$ for every k . The naive algorithm is $O(N^2)$; despite decades of effort, no one has beaten this by a polynomial factor. The *hypothesis* is that $\Omega(N^{2-o(1)})$ is actually necessary.
- This is the same style of assumption as 3SUM-hardness or SETH – a widely-believed barrier used to explain *why* a whole family of problems seems stuck at quadratic time.

Proof idea: knapsack *is* a convolution in disguise

- Recall the knapsack recurrence: $M[i, c] = \max(M[i-1, c], v_i + M[i-1, c - w_i])$ – at each capacity c , you take the better of “skip item i ” and “take item i .”
- Group items by weight; merging the DP table across a block of items with the *same* weight is exactly a $(\min, +)$ -style combination of two “profile” arrays (one array’s best value at each residual capacity, combined with another).
- The 2017 reduction makes this precise in the other direction: it builds a knapsack instance out of two arbitrary $(\min, +)$ -convolution inputs, such that *any* algorithm solving that knapsack instance in $o((n + w_{\max})^2)$ time would also solve $(\min, +)$ -convolution in $o(N^2)$ time – contradicting the hypothesis.
- Takeaway: the “quadratic wall” isn’t a failure of imagination in this week’s DP – it is now tied to one of the deepest open barriers in fine-grained complexity.

The race to close the gap (2017–2024)

Algorithm	Time (in $n, w_{\max}$)	Source
This week's folklore DP	$O(n w_{\max})$	standard textbook DP
Polak, Rohwedder & Węgrzycki	$O(n + w_{\max}^3)$	2021
Chen, Lian, Mao & Zhang	$\tilde{O}(n + w_{\max}^{12/5})$	2023
Jin	$O(n + w_{\max}^2 \log^4 w_{\max})$	STOC 2024

- Jin's 2024 result ("0-1 Knapsack in Nearly Quadratic Time," STOC 2024, arXiv:2308.04093) is deterministic and matches the 2017 conditional lower bound up to only a $\text{polylog}(w_{\max})$ factor – the gap is essentially closed.
- This is a genuine, seven-year, multi-paper research arc: a believable hardness *hypothesis* (2017) followed by a sequence of algorithms that chase the matching upper bound down to almost exactly that wall (2021 $\rightarrow$ 2023 $\rightarrow$ 2024).

A silent assumption in this week's DP

- Weighted Interval Scheduling's $O(n \log n)$ DP – and Knapsack's $O(nW)$ DP – are both **offline**: every interval/item, with its weight, is known before the algorithm starts.
- Real schedulers (a cloud job queue, an ad-auction system, a CPU scheduler) must commit to decisions **online**, as requests arrive, without knowing what is coming next. The right question changes from “what is the optimum?” to “how close to optimum can *any* algorithm get without seeing the future?” – a **competitive ratio**.

Warm-up: the ski rental problem

- Renting skis costs \$1/day; buying costs $\$B$ once, after which every day is free. You don't know in advance how many days d you will ski.
- **Strategy:** rent for the first $B - 1$ days, then buy on day B if you're still skiing.
- **Worked example, $B = 10$:** if $d = 9$, you pay exactly 9 (never buy) – optimal. If $d = 10$, you pay 9 (renting) + 10 (buying on day 10) = 19, versus the optimal-with-hindsight cost of 10 – a ratio of 1.9, approaching the worst-case ratio of 2 as B grows.
- This deterministic strategy is provably **2-competitive**, and no deterministic algorithm without extra information can do better in the worst case.

Beating the worst case with a prediction

Purohit, Svitkina & Kumar, “Improving Online Algorithms via ML Predictions,” NeurIPS 2018

Suppose an ML model predicts $\hat{d}$, the number of days you'll ski. Buy on day $\min(\hat{d}, B)$ instead of always waiting until day B .

- If the prediction is **accurate** ($\hat{d} \approx d$), this strategy's cost approaches the *optimal-with-hindsight* cost – far better than the worst-case ratio of 2.
- If the prediction is **wildly wrong**, a tunable parameter $\lambda \in [0, 1]$ trades off how much you trust it, guaranteeing the cost never exceeds a bounded multiple of the 2-competitive baseline – the algorithm degrades gracefully instead of failing catastrophically.
- The same paper applies this idea to **non-clairvoyant job scheduling** – exactly this week's scheduling setting, but online and with a predicted job length replacing the “ w_j , known in advance” assumption of the offline DP.

Formalizing the trade-off: consistency vs. robustness

Mitzenmacher & Vassilvitskii, “Algorithms with Predictions,” Communications of the ACM, 2022

Every learning-augmented algorithm should be measured on two axes:

- **Consistency:** performance when the prediction is perfect (ideally matching the offline optimum).
 - **Robustness:** performance when the prediction is adversarially wrong (ideally still bounded by the classical worst-case guarantee).
-
- This framework, formalized in 2018–2022 and still an active research area through 2024–2026, is now applied to scheduling, caching, bloom filters, and – yes – knapsack-style packing problems: exactly the offline problems this week introduced, now redesigned to gracefully use (possibly imperfect) predictions about the future instead of assuming perfect foreknowledge.

Summary

- 1 **The $O(nW)$ pseudo-polynomial bound is not an accident of laziness.** A 2017 conditional lower bound (Cygan–Mucha–Węgrzycki–Włodarczyk; Künnemann–Paturi–Schneider) ties any substantial improvement to breaking the $(\min, +)$ -convolution hardness hypothesis.
- 2 **The gap is now almost closed.** A sequence of papers (2021, 2023, and Jin’s STOC 2024 result) pushed the true *upper* bound down to $O(n + w_{\max}^2 \log^4 w_{\max})$ – matching the lower bound up to polylog factors.
- 3 **The offline assumption itself is now optional.** Learning-augmented algorithms (Purohit–Svitkina–Kumar 2018; formalized by Mitzenmacher–Vassilvitskii 2022) let this week’s scheduling ideas work online, using ML predictions with provable consistency/robustness guarantees.

Looking ahead

Next week (Dynamic Programming II) applies the same five-step DP recipe to sequence alignment and shortest paths – and, as you’ll see, edit distance’s $O(nm)$ bound has its *own* fine-grained lower bound story.